Signup and get free access to 100+ Tutorials and Practice Problems Start Now

8 LIVE EVENTS

← Notes

Dynamic Programming - Problems involving Grids

151 Dynamic Programming Competitive Programming Grid

1. Introduction

There are many problems in online coding contests which involve finding a minimum-cost path in a grid, finding the number of ways to reach a particular position from a given starting point in a 2-D grid and so on. This post attempts to look at the dynamic programming approach to solve those problems. The problems which will be discussed here are :

> 1. Finding the Minimum Cost Path in a Grid when a Cost Matrix is given.
> 2. Finding the number of ways to reach from a starting position to an ending position travelling in specified directions only.
> 3. Finding the number of ways to reach a particular position in a grid from a starting position (given some cells which are blocked)

---

2. Finding Minimum-Cost Path in a 2-D Matrix

> **Problem Statement** : Given a cost matrix Cost[][] where Cost[i][j] denotes the Cost of visiting cell with coordinates (i,j), find a min-cost path to reach a cell (x,y) from cell (0,0) under the condition that you can only travel one step right or one step down. (We assume that all costs are positive integers)

Solution : It is very easy to note that if you reach a position (i,j) in the grid, you must have come from one cell higher, i.e. (i-1,j) or from one cell to your left , i.e. (i,j-1). This means that the cost of visiting cell (i,j) will come from the following recurrence relation:

```
MinCost(i,j) = min(MinCost(i-1,j),MinCost(i,j-1)) + Cost[i][j]
```

The above statement means that to reach cell (i,j) wit minimum cost, first reach either cell(i-1,j) or cell (i,j-1) in as minimum cost as possible. From there, jump to cell (i,j). This brings us to the two important conditions which need to be satisfied for a dynamic programming problem:

?

> **Optimal Sub-structure**:- Optimal solution to a problem involves optimal solutions to sub-problems.
> **Overlapping Sub-problems**:- Subproblems once computed can be stored in a table for further use. This saves the time needed to compute the same sub-problems again and again.

8 LIVE EVENTS

(You can google the above two terms for more details)

The problem of finding the min-Cost Path is now almost solved. We now compute the values of the base cases: the topmost row and the leftmost column. For the topmost row, a cell can be reached only from the cell on the left of it. Assuming zero-based index,

```
MinCost(0,j) = MinCost(0,j-1) + Cost[0][j]
```

i.e. cost of reaching cell (0,j) = Cost of reaching cell (0,j-1) + Cost of visiting cell (0,j) Similarly,

```
MinCost(i,0) = MinCost(i-1,0) + Cost[i][0]
```

i.e. cost of reaching cell (i,0) = Cost of reaching cell (i-1,0) + Cost of visiting cell (i,0)

Other values can be computed from them. See the code below for more understanding.

```
#include <bits/stdc++.h>
using namespace std;
#define F(i,a,b) for(int i = (int)(a); i <= (int)(b); i++)
#define RF(i,a,b) for(int i = (int)(a); i >= (int)(b); i--)
int main()
{
    int X,Y; //X:number of rows, Y: number of columns
    X = Y = 10; //assuming 10X10 matrix
    int Cost[X][Y];

    F(i,0,X-1)
    {
        F(j,0,Y-1)
        {
            //Take input the cost of visiting cell (i,j)
            cin>>Cost[i][j];
        }
    }
```

?

```
    int MinCost[X][Y]; //declare the minCost matrix

    MinCost[0][0] = Cost[0][0];

    // initialize first row of MinCost matrix
    F(j,1,Y-1)
        MinCost[0][j] = MinCost[0][j-1] + Cost[0][j];

    //Initialize first column of MinCost Matrix
    F(i,1,X-1)
        MinCost[i][0] = MinCost[i-1][0] + Cost[i][0];

    //This bottom-up approach ensures that all the sub-problems needed
    // have already been calculated.
    F(i,1,X-1)
    {
        F(j,1,Y-1)
        {
            //Calculate cost of visiting (i,j) using the
            //recurrence relation discussed above
            MinCost[i][j] = min(MinCost[i-1][j],MinCost[i][j-1]) + Cost[i]
[j];
        }
    }

    cout<<"Minimum cost from (0,0) to (X,Y) is "<<MinCost[X-1][Y-1];
    return 0;
}
```

Another variant of this problem includes another direction of motion, i.e. one is also allowed to move diagonally lower from cell (i,j) to cell (i+1,j+1). This question can also be solved easily using a slight modification in the recurrence relation. To reach (i,j), we must first reach either (i-1,j), (i,j-1) or (i-1,j-1).

```
MinCost(i,j) = min(MinCost(i-1,j),MinCost(i,j-1),MinCost(i-1,j-1)) + Cost[i][j]
```

---

2. Finding the number of ways to reach from a starting position to an ending position travelling in specified directions only.

8 LIVE EVENTS

?

> **Problem Statement** : Given a 2-D matrix with M rows and N columns, find the number of ways to reach cell with coordinates (i,j) from starting cell (0,0) under the condition that you can only travel one step right or one step down.

8 LIVE EVENTS

Solution : This problem is very similar to the previous one. To reach a cell (i,j), one must first reach either the cell (i-1,j) or the cell (i,j-1) and then move one step down or to the right respectively to reach cell (i,j). After convincing yourself that this problem indeed satisfies the optimal sub-structure and overlapping subproblems properties, we try to formulate a bottom-up dynamic programming solution.

We first need to identify the states on which the solution will depend. To find the number of ways to reach to a position, what are the variables on which my answer depends? Here, we need the row and column number to uniquely identify a position. For more details on how to decide the state of a dynamic programming solution, see this : How can one start solving Dynamic Programming problems? Therefore, let NumWays(i,j) be the number of ways to reach position (i,j). As stated above, number of ways to reach cell (i,j) will be equal to the sum of number of ways of reaching (i-1,j) and number of ways of reaching (i,j-1). Thus, we have our recurrence relation as :

```
numWays(i,j) = numWays(i-1,j) + numWays(i,j-1)
```

Now, all you need to do is take care of the base cases and the recurrence relation will calculate the rest for you. :)

The base case, as in the previous question, are the topmost row and leftmost column. Here, each cell in topmost row can be visited in only one way, i.e. from the left cell. Similar is the case for the leftmost column. Hence the code is:

```
#include <bits/stdc++.h>
using namespace std;
#define F(i,a,b) for(int i = (int)(a); i <= (int)(b); i++)
#define RF(i,a,b) for(int i = (int)(a); i >= (int)(b); i--)
int main()
{
    int X,Y; //X:number of rows, Y: number of columns
    X = Y = 10; //assuming 10X10 matrix

    int NumWays[X][Y]; //declare the NumWays matrix
    NumWays[0][0] = 1;
    // initialize first row of NumWays matrix
    F(j,1,Y-1)
        NumWays[0][j] = 1;
```

?

```
    //Initialize first column of NumWays Matrix
    F(i,1,X-1)
        NumWays[i][0] = 1;
    //This bottom-up approach ensures that all the sub-problems needed
    // have already been calculated.
    F(i,1,X-1)
    {
        F(j,1,Y-1)
        {
            //Calculate number of ways visiting (i,j) using the
            //recurrence relation discussed above
            NumWays[i][j] = NumWays[i-1][j] + NumWays[i][j-1];
        }
    }

    cout<<"Number of ways from(0,0) to (X,Y) is "<<NumWays[X-1][Y-1];
    return 0;
}
```

8 LIVE EVENTS

---

3. Finding the number of ways to reach a particular position in a grid from a starting position (given some cells which are blocked)

> **Problem Statement** : You can read the problem statement here: Robots and Paths Input is three integers M, N and P denoting the number of rows, number of columns and number of blocked cells respectively. In the next P lines, each line has exactly 2 integers i and j denoting that the cell (i, j) is blocked.

Solution : The code below explains how to proceed with the solution. The problem is same as the previous one, except for few extra checks(due to blocked cells.)

```
#include <bits/stdc++.h>
using namespace std;
typedef long long int ll;
#define F(i,a,b) for(int i = (int)(a); i <= (int)(b); i++)
#define RF(i,a,b) for(int i = (int)(a); i >= (int)(b); i--)
#define MOD 1000000007
int main()
{
    int M,N,P,_i,_j;
```

?

```
    //Take input the number of rows, columns and blocked cells
    cin>>M>>N>>P;

    //declaring a Grid array which stores the number of paths
    ll Grid[M+1][N+1];

    //Note that we'll be using 1-based indexing here.
    //initialize all paths initially as 0
    memset(Grid, 0, sizeof(Grid));

    F(i,0,P-1)
    {
        //Take in the blocked cells and mark them with a special value(-1
here)
        cin>>_i>>_j;
        Grid[_i][_j] = -1;
    }

    // If the initial cell is blocked, there is no way of moving anywhere
    if(Grid[1][1] == -1)
    {
        printf("0");
        return 0;
    }

    // Initializing the leftmost column
    //Here, If we encounter a blocked cell, there is no way of visiting any
cell
    //directly below it.(therefore the break)
    F(i,1,M)
    {
        if(Grid[i][1] == 0) Grid[i][1] = 1LL;
        else break;
    }

    //Similarly initialize the topmost row.
    F(i,2,N)
    {
        if(Grid[1][i] == 0) Grid[1][i] = 1LL;
        else break;
    }
```

8
LIVE EVENTS
?

```
    //Now the recurrence part
    //The only difference is that if a cell has been marked as -1,
    //simply ignore it and continue to the next iteration.
    F(i,2,M)
    {
        F(j,2,N)
        {
            if(Grid[i][j] == -1) continue;

            //While adding the number of ways from the left and top cells,
            //check that they are reachable,i.e. they aren't blocked

            if(Grid[i-1][j] > 0) Grid[i][j] = (Grid[i][j] + Grid[i-1][j] +
MOD)%MOD;
            if(Grid[i][j-1] > 0) Grid[i][j] = (Grid[i][j] + Grid[i][j-1] +
MOD)%MOD;
        }
    }

    //If the final cell is blocked, output 0, otherwise the answer
    printf("%lld",(Grid[M][N] >= 0 ? Grid[M][N] : 0));
    return 0;
}
```

Another variant

Finally, we discuss another variant of problems involving grids. You can see the problem here. Working Out A short description of the problem is:

> **Problem Statement** : You are given a 2-D matrix A of n rows and m columns where A[i][j] denotes the calories burnt. Two persons, a boy and a girl, start from two corners of this matrix. The boy starts from cell (1,1) and needs to reach cell (n,m). On the other hand, the girl starts from cell (n,1) and needs to reach (1,m). The boy can move right and down. The girl can move right and up. As they visit a cell, the amount in the cell A[i][j] is added to their total of calories burnt. You have to maximize the sum of total calories burnt by both of them under the condition that they shall meet only in one cell and the cost of this cell shall not be included in either of their total.

Solution : Let us analyse this problem in steps:

8 LIVE EVENTS

?

> The boy can meet the girl in only one cell.

So, let us assume they meet at cell (i,j).

Boy can come in from left or the top, i.e. (i,j-1) or (i-1,j). Now he can move right or down.That is, the sequence for the boy can be:

```
(i,j-1)-->(i,j)-->(i,j+1)
(i,j-1)-->(i,j)-->(i+1,j)
(i-1,j)-->(i,j)-->(i,j+1)
(i-1,j)-->(i,j)-->(i+1,j)
```

Similarly, the girl can come in from the left or bottom, i.e. (i,j-1) or (i+1,j) and she can go up or right. The sequence for girl's movement can be:

```
(i,j-1)-->(i,j)-->(i,j+1)
(i,j-1)-->(i,j)-->(i-1,j)
(i+1,j)-->(i,j)-->(i,j+1)
(i+1,j)-->(i,j)-->(i-1,j)
```

Comparing the 4 sequences of the boy and the girl, the boy and girl meet only at one position (i,j), iff

```
Boy: (i,j-1)-->(i,j)-->(i,j+1) and Girl: (i+1,j)-->(i,j)-->(i-1,j)
```

or

```
Boy: (i-1,j)-->(i,j)-->(i+1,j) and Girl:  (i,j-1)-->(i,j)-->(i,j+1)
```

Convince yourself that in no other case will they meet at only one position.

Now, we can solve the problem by creating 4 tables:

1. Boy's journey from start (1,1) to meeting cell (i,j)
2. Boy's journey from meeting cell (i,j) to end (n,m)
3. Girl's journey from start (n,1) to meeting cell (i,j)
4. Girl's journey from meeting cell (i,j) to end (1,n)

The meeting cell can range from `2<= i <= n-1 and 2 <= j <= m-1`

See the code below for more details:

8
LIVE EVENTS
?

```
#include <bits/stdc++.h>
#define F(i,a,b) for(int i = (int)(a); i <= (int)(b); i++)
#define RF(i,a,b) for(int i = (int)(a); i >= (int)(b); i--)
#define MAX 1005
int Boy1[MAX][MAX];
int Boy2[MAX][MAX];
int Girl1[MAX][MAX];
int Girl2[MAX][MAX];
using namespace std;
int main()
{
    int N,M,ans,op1,op2;
    scanf("%d%d",&N,&M);
    int Workout[MAX][MAX];
    ans = 0;

    //Take input the calories burnt matrix
    F(i,1,N)
        F(j,1,M)
            scanf("%d",&Workout[i][j]);

    //Table for Boy's journey from start to meeting cell
    F(i,1,N)
        F(j,1,M)
            Boy1[i][j] = max(Boy1[i-1][j],Boy1[i][j-1]) + Workout[i][j];

    //Table for boy's journey from end to meet cell
    RF(i,N,1)
        RF(j,M,1)
            Boy2[i][j] = max(Boy2[i+1][j],Boy2[i][j+1]) + Workout[i][j];

    //Table for girl's journey from start to meeting cell
    RF(i,N,1)
        F(j,1,M)
            Girl1[i][j] = max(Girl1[i+1][j],Girl1[i][j-1]) + Workout[i][j];

    //Table for girl's journey from end to meeting cell
    F(i,1,N)
        RF(j,M,1)
            Girl2[i][j] = max(Girl2[i-1][j],Girl2[i][j+1]) + Workout[i][j];

    //Now iterate over all meeting positions (i,j)
```

8
LIVE EVENTS
?

```
    F(i,2,N-1)
    {
        F(j,2,M-1)
        {
            //For the option 1
            op1 = Boy1[i][j-1] + Boy2[i][j+1] + Girl1[i+1][j] + Girl2[i-1]
[j];

            //For the option 2
            op2 = Boy1[i-1][j] + Boy2[i+1][j] + Girl1[i][j-1] + Girl2[i]
[j+1];

            //Take the maximum of two options at each position
            ans = max(ans,max(op1,op2));
        }
    }

    printf("%d",ans);
    return 0;
}
```

Thanks for reading. :) I hope it was helpful!

Like 21 Tweet

8 LIVE EVENTS

COMMENTS (41) SORT BY: Relevance

Login/Signup to Comment

Shekhar Prasad Rajak 4 years ago
thanks .. :)
2 votes ● Reply ● Message ● Permalink

Rajan Kasodariya 4 years ago
Useful notes..Thanks
1 vote ● Reply ● Message ● Permalink

Jayant Jaiswal 4 years ago
In the last question, can't the meeting position of the boy and girl be
Boy:(i-1,j)-->(i,j)-->(i+1,j) and Girl:(i+1,j)-->(i,j)-->(i-1,j)
????
0 votes ● Reply ● Message ● Permalink

Shubham Goswami Author 4 years ago
In that case, the boy and girl visit 3 same cells. Let us suppose that they follow this path. Then we can have another path for either the boy or the girl which contains two cells

which are still not visited and thus can gain more points. Therefore they will never choose such kind of paths.

0 votes • Reply • Message • Permalink

Anuj Thakur 4 years ago

can you please elaborate this point ,if possible with diagram.

0 votes • Reply • Message • Permalink

Palash Bansal 3 years ago

As, it is not mentioned that, when a cell they visit, cell's value doesn't decreases. And so, at different time, they can visit same cell to get their calories burnt value increase. So, these cases can be possible-
Boy: (i,j-1)-->(i,j)-->(i+1,j) and Girl: (i+1,j)-->(i,j)-->(i-1,j) i.e b: right->down g: up->up
Boy: (i,j-1)-->(i,j)-->(i+1,j) and Girl: (i+1,j)-->(i,j)-->(i,j+1) i.e. b: right->down g: up->right
and so on..
Please clarify.

0 votes • Reply • Message • Permalink

ANKUSH PATEL 3 years ago

Same doubt! #following

0 votes • Reply • Message • Permalink

Shubham Goswami Author 3 years ago

Your doubt is again same as Kushagra's.
Please read the entire description of the problem here :
http://codeforces.com/contest/429/problem/B
It says that : "They have to meet in exactly one cell of gym. Note, that Iahub and Iahubina can perform workouts with different speed, so the number of cells that they use to reach meet cell may differs." So there can be only one common meeting cell.

0 votes • Reply • Message • Permalink

Arneish Prateek 8 months ago

Your explanation doesn't make sense. I do not see how that paragraph you quote answers this doubt.

0 votes • Reply • Message • Permalink

Rohan Passi 2 years ago

Please refer contest announcement, not problem statement.

0 votes • Reply • Message • Permalink

Kushagra Jain Edited 3 years ago

100 100 100
100 1 100
100 1000 100

consider the above matrix, wont the answer be maximum when they both meet at 1 and also visit the cell with 1000 value? they are visiting 2 same cells, but meeting only once. This gives us max value of 2600 however your function return 1700 as the answer.
Please correct me if I am wrong.

1 vote • Reply • Message • Permalink

Shubham Goswami Author 3 years ago

Please read the entire description of the problem here :
http://codeforces.com/contest/429/problem/B
It says that : "They have to meet in exactly one cell of gym. Note, that Iahub and Iahubina can perform workouts with different speed, so the number of cells that

8 LIVE EVENTS

they use to reach meet cell may differs." So there can be only one common meeting cell.
0 votes • Reply • Message • Permalink

Kushagra Jain 3 years ago
Oh sorry, my bad! Thanks for the clarification.
0 votes • Reply • Message • Permalink

Khushbu Jain 3 years ago
what are boy2 and girl 2 matrices for? boy can not go up and left..and girl can't go down and left? confused
1 vote • Reply • Message • Permalink

Shubham Goswami Author 3 years ago
Those matrices cover their journey from end to meet cell. See this line : Boy2[i][j] = max(Boy2[i+1][j],Boy2[i][j+1]) + Workout[i][j]; This means that Boy2[i][j] stores the most optimal value for Boy till cell (i,j) assuming he came from end by following the reversed path. i.e. he will move right and down only, but we calculate the right and down movement part first.
0 votes • Reply • Message • Permalink

Khushbu Jain 3 years ago
thanks
0 votes • Reply • Message • Permalink

Sangeeth Konumuri 4 years ago
Very good presentation.. Thank You!!
0 votes • Reply • Message • Permalink

Rishav Sharma 4 years ago
a great tutorial indeed.. :)
0 votes • Reply • Message • Permalink

satishkumar 4 years ago
good tutorial. anybody else can have.
0 votes • Reply • Message • Permalink

feature sky 4 years ago
Please help me Understanding this:
Grid[i][j] = (Grid[i][j] + Grid[i-1][j]
why we are adding Grid[i][j] with Grid[i-1][j] ??
0 votes • Reply • Message • Permalink

Shubham Goswami Author 4 years ago
We add the value in Grid[i-1][j] to the value in Grid[i][j], i.e. we don't want to lose out the previous value (if present) in Grid[i][j].
0 votes • Reply • Message • Permalink

Setty Venkata Sai Meghana 4 years ago
Boy:(i,j-1)-->(i,j)-->(i,j+1) and Girl: (i+1,j)-->(i,j)-->(i,j+1)
In this the boy and girl are visiting the common cells twice(i,j) and (i,j+1)
Is that correct?? Did i miss anywhere?? Thanks in advance.
0 votes • Reply • Message • Permalink

Shubham Goswami Author 4 years ago
Yep. you got it right! Corrected it in the note. Thanks for pointing it out!!
0 votes • Reply • Message • Permalink

8 LIVE EVENTS
?

Geeta Sharma 4 years ago

RF(i,N,1)
RF(j,M,1)
Boy2[i][j] = max(Boy2[i+1][j],Boy2[i][j+1]) + Workout[i][j];
where we get Boy2[i+1] from??

0 votes • Reply • Message • Permalink

Yash Sharma 4 years ago

reaching [n][m] from [1][1] moving in right and down direction by x and y respectively that is either right or down or both
by any distance ..?

0 votes • Reply • Message • Permalink

MUKESH KUMAR GUPTA 3 years ago

WOW! NICE WORK MATE... I M NOVICE CODER... THE WAY U EXPLAINED IS AWESOME. THANKS A LOT.
HAPPY READING.

0 votes • Reply • Message • Permalink

Shailesh Singh 3 years ago

Thank you..!!

0 votes • Reply • Message • Permalink

RAJAT MODI 3 years ago

very good tutorial really aprreciate it .

0 votes • Reply • Message • Permalink

Mandar Kakade 3 years ago

Thx!

0 votes • Reply • Message • Permalink

Paras Jain 3 years ago

Thanks a lot. You've covered it quite thoroughly. Keep on posting :D

0 votes • Reply • Message • Permalink

RISHAV KUMAR 3 years ago

why can't i=1 be included in the meeting region ? Like girl starts moving up and boy starts moving down n they just cross each other ?

0 votes • Reply • Message • Permalink

Shubham Goswami Author 3 years ago

Just try and simulate it. This is not possible due to the constraint that there has to be only a single meeting cell.

0 votes • Reply • Message • Permalink

Rahul Goyal 3 years ago

very good explanation ...!!

0 votes • Reply • Message • Permalink

Anup Verma 3 years ago

Regarding Problem 3:
What is the use of MOD here:
if(Grid[i-1][j] > 0) Grid[i][j] = (Grid[i][j] + Grid[i-1][j] + MOD)%MOD;
if(Grid[i][j-1] > 0) Grid[i][j] = (Grid[i][j] + Grid[i][j-1] + MOD)%MOD;

0 votes • Reply • Message • Permalink

8 LIVE EVENTS

?

Shubham Goswami Author Edited 3 years ago

This is done to avoid overflowing of value beyond MOD. It is a requirement of the question. "Since the answer can get very large, output the answer modulo 1000000007."
See here : https://www.codechef.com/problems/CD1IT4
This also might be helpful : http://stackoverflow.com/questions/1082917/mod-of-negative-number-is-melting-my-brain

0 votes • Reply • Message • Permalink

Sonakshi Mehra Edited 2 years ago

if(Grid[i-1][j] > 0) Grid[i][j] = (Grid[i][j] + Grid[i-1][j] + MOD)%MOD;
if(Grid[i][j-1] > 0) Grid[i][j] = (Grid[i][j] + Grid[i][j-1] + MOD)%MOD;

I get why you are doing %MOD to the total sum, but why add MOD? I mean why (Grid[i][j] + Grid[i-1][j] + MOD)%MOD instead of (Grid[i][j] + Grid[i-1][j])%MOD ???

0 votes • Reply • Message • Permalink

nikhil ranjan 3 years ago

where could i find similar dp notes?

0 votes • Reply • Message • Permalink

Isha Agarwal 2 years ago

Very useful. Thanks!

0 votes • Reply • Message • Permalink

TARUN GULATI 2 years ago

very useful ... thanks

0 votes • Reply • Message • Permalink

Badal Agrawal a year ago

Why arent we adding workout[i][j] in final op1 and op2?

0 votes • Reply • Message • Permalink

suchismith roy Edited 7 months ago

Your explanation for the cases, when boy comes from up..and girl comes from down is still not clear. Nowhere in the problem has it been described that they cant visit the same cell again. Is it that when one visits an already visited cell, the calories burnt is not counted? Could you kindly explain this one case elaborately.

"If a workout was done by either Iahub or Iahubina, it counts as total gain." Does this mean that if both do the workout, it is not counted ?

0 votes • Reply • Message • Permalink

8 LIVE EVENTS

AUTHOR

Shubham Goswami
Software Engineer at Sams...
Bangalore, Karnataka, India
1 note

TRENDING NOTES

Python Diaries Chapter 3 Map | Filter | For-else | List Comprehension

?

written by Divyanshu Bansal

Bokeh | Interactive Visualization Library | Use Graph with Django Template
written by Prateek Kumar

Bokeh | Interactive Visualization Library | Graph Plotting
written by Prateek Kumar

Python Diaries chapter 2
written by Divyanshu Bansal

Python Diaries chapter 1
written by Divyanshu Bansal

more ...

8
LIVE EVENTS